弹幕之光:2D 图形加速系统

摘要

本作品为解决嵌入式图形界面与高密度二维游戏中像素绘制占用大量 CPU 时间的问题，基于易灵思 Titanium Ti60F225 FPGA 与 Sapphire RISC-V 软核，设计实现了一套软硬件协同的二维图形渲染加速系统。系统由 RISC-V 处理器运行 C 语言游戏逻辑，通过 APB3 总线配置硬件并提交绘制命令；FPGA 端的 BitBLT 引擎通过 128 位 AXI 总线访问 DDR3，完成图像块拷贝、纯色填充、颜色键透明处理、Alpha 混合及饱和加法，将底层像素搬运与混合操作交由专用硬件执行。

针对高密度弹幕与动态特效的显示需求，系统构建了基于扫描线的弹幕绘制与多图层实时合成逻辑，将背景、弹幕、天气、激光和程序化光环进行合成，并通过 Bloom 泛光与色彩变换增强视觉表现。软件结合缓存维护、双帧缓冲及配对桶表完成数据交接，显示端支持 1366×768 分辨率、75 Hz 刷新率，经 HDMI 接口输出 DVI 兼容视频信号。作品以三阶段弹幕游戏和高负载压力场景作为验证载体，并在压力场景中实现运动更新、碰撞候选筛选与弹幕分桶的硬件加速，展示 RISC-V 与 FPGA 在控制、计算和显示任务中的协同能力。在相同场景与显示配置下，硬件加速模式的平均帧率由 25 FPS 提升至 75 FPS，并在 20000 个活动弹幕条件下保持 75 FPS。帧率及弹幕计算方式及代码见 1.5 主要性能指标。

第一部分　作品概述

1.1 功能与特性

系统输出 1366×768 的画面，帧缓冲像素为 ARGB8888。帧缓冲有两块，地址分别为 0x00800000 和 0x00E00000，显示读走当前块，BitBLT 写准备块，场同步附近再交换基址。扫描读把一行分成 684 像素和其余 682 像素两段，两段可以指向不同基址；菜单、关卡和压力测试把两段基址写成同一个缓冲，送出一整幅画面，BitBLT 对比画面则让左右两段分别指向各自的缓冲。BitBLT 有四种操作：纯色填充、矩形拷贝、按全局 Alpha 与颜色键混合，以及按通道饱和的加法。显示引擎依次叠加帧缓冲底图、天气粒子、弹幕、激光、冲击环与符卡光束、自机光束与预警线、HUD，再做泛光和 3×3 色彩矩阵。色彩矩阵包含

直通、闪白、偏红、灰度、反相、偏绿和偏蓝。HUD 上的帧率由硬件统计扫描基址在 1 秒内的变化次数，固件读取 APB 的 BCD 寄存器后画出，最大显示 99。I2C 手柄提供两轴 0～255 的方位数据和五个按键(包含手柄的中心按压按键)。音频走 I2S 通信协议 ，外接 MAX98357A 功放模块播放游戏 BGM（上海アリス幻樂団 - シンデレラケージ ～ Kagome-Kagome）与打击音效。演示内容是一套纵版射击游戏，菜单、关卡背景、自机(玩家)、敌机和压力测试等都调用上述硬件。

1.2 游戏功能与操作

上电后进入主菜单，屏幕上有两张卡片。RENDER LAB 进入 CPU 与 FPGA 对比界面：左半屏由 CPU 逐像素绘制，右半屏由 BitBLT 绘制同一组图形，两侧分别显示帧率。START GAME 进入三关纵版射击，关卡依次为冰、火和梦。在主菜单按住 A 或 B 再确认，进入压力关；按住 Y 再确认，进入精灵压力演示。设置菜单里的 STRESS STAGE 也可以进入压力关。

菜单用摇杆上下选择，Y 向上，A 向下，X 或摇杆中键确认。局内摇杆移动自机，按住 A 减速。X 射击。击破敌机可以掉落回复、射速、攻击、武器和得分，武器按 FOCUS、SPREAD、LASER、FAN 逐级解锁。击破敌人会积累符卡，蓄满后按摇杆中键释放。BOSS 出现前有对话，按 X 或摇杆中键翻过当前句。B 暂停并打开设置，项目为无限复活、硬件加速、继续游戏、退出到菜单、进入压力关、音量和泛光。设置里同样用 Y、A 移动，X 或摇杆中键确认。

压力关保留移动和射击。按住 Y 提高 LOAD，每次增加 128，最高 30000；按住 A 降低 LOAD，最低回到初始的 192。画面上的 SPRITES 是当前存活对象数，DROP 是这一档以来的丢弃峰值，不是累计发射量。对比界面没有暂停菜单，按 B 回到主菜单。板载 KEY0 和 KEY2 在关卡、压力关、对比界面和精灵压力演示中都直接回到主菜单。

1.3 应用领域

本作品主要面向嵌入式图形界面、高密度二维游戏及图形加速等应用领域。在嵌入式人机交互场景中，可用于仪器仪表、设备控制面板等界面的图块更新、

图标叠加与透明效果绘制，将重复性的像素搬运和混合操作交由 FPGA 执行，为处理器保留更多应用控制与数据处理时间。在二维游戏和交互演示场景中，系统支持精灵绘制、弹幕显示及动态图层合成，可用于弹幕射击、粒子动画和多特效交互展示。通过通用图块绘制与扫描线合成相结合，满足大量动态元素的显示需求。此外，本作品可作为 RISC-V 与 FPGA 软硬件协同设计的实践平台，支持总线接口、存储访问、双缓冲、图像混合及视频时序等实验，也可用于定制图形加速算法的原型验证与性能对比。

1.4 主要技术特点

本作品采用 RISC-V 与 FPGA 异构架构，实现应用控制与图形处理的协同分工。Sapphire RISC-V 软核运行游戏逻辑，通过 APB3 总线提交绘制命令和配置参数；FPGA 端完成图块拷贝、纯色填充、颜色键透明处理、Alpha 混合及饱和加法，降低处理器承担的像素操作负担。系统通过 128 位 AXI 总线访问 DDR3，结合突发传输与内部 FIFO 缓存提高连续图像数据的传输效率，并通过缓存维护、双帧缓冲和配对桶表协调处理器、加速器与显示模块之间的数据交接，避免显示过程中读写同一帧缓冲造成画面撕裂。针对高密度弹幕与动态特效，系统将通用 BitBLT 绘制与扫描线实时合成相结合。弹幕、天气、激光及程序化光环在显示过程中生成并参与合成，减少相关图层逐帧写入显存的开销，同时支持 Bloom 泛光和色彩变换，输出 1366×768 分辨率、75 Hz 的画面。系统通过寄存器和数据描述符调整图层及特效参数，支持在不修改 RTL 代码的情况下修改软件表现；在压力场景中进一步引入运动更新、碰撞候选筛选与弹幕分桶的硬件加速，并提供软件与硬件渲染对比及运行状态显示，便于验证加速效果和分析性能瓶颈。

1.5 主要性能指标

本作品的主要性能指标如下表所示：

指标项目	设计指标与实测参数

目标芯片平台 (FPGA)	Titanium Ti60F225I3
嵌入式软核处理器 (CPU)	Efinix Sapphire RISC-V RV32IM 软核
片上极速内存 (TCM)	64 KiB BRAM（0xF9000000）
板载显存规格 (DDR3 SDRAM)	256 MiB，16 位数据线；
系统总线互联架构	AXI4 128 位@100MHz；APB3 外设总线
视频输出接口与物理协议	HDMI 输出,DVI 兼容 TMDS 差分
显示输出分辨率与刷新率	1366*768,75Hz
系统时钟频率体系	core_clk 100MHz； HDMI 输出时钟 85.71MHz； DDR3 时钟 400MHz； TMDS 串行化 428.6MHz
活动弹幕最大并发容量	压力测试模式最大 20000 颗 (26*26) 注:代码实际最高容量 30000 颗
系统图形渲染帧率	75FPS
音频处理与外部功放规格	I2S 驱动 MAX98357A 数字功放
人机交互控制接口	I2C 手柄（两轴 0～255、5 个按键）、 板载 GPIO 按键、UART 调试串口

表 1-1：主要性能指标

Core Resources	
Inputs	58 / 1703
Outputs	252 / 2267
XLRs	58982 / 60800
Clocks	10 / 64
Memory Blocks	256 / 256
DSP Blocks	107 / 160

图 1-1 核心资源占用

Periphery Resource	
GPIO	15 / 23
HSIO	32.0 / 70
JTAG User TAP	1 / 4
Oscillator	0 / 1
PLL	3 / 4

图 1-2 外设资源占用

Timing (Final)	
Worst Negative Slack (WNS)	0.364 ns
Worst Hold Slack (WHS)	0.026 ns
core_clk	109.673 MHz
sdram_clk	476.644 MHz
rx_cal_clk	479.616 MHz
tx_cal_clk	469.704 MHz
jtag_inst1_TCK	236.911 MHz
tx_cal_clk_90edge	468.165 MHz
hdmi_tx_slow_clk	91.929 MHz

图 1-3 时序收敛结果

弹幕及帧率计算方法：

菜单第一项进入 CPU 与 FPGA 对比界面。固件在 app 子文件夹的 main.c 中只在这一屏打开硬件 OSD：左半屏由 CPU 逐像素填写，整幅完成后才向左路扫描基址提交一次；右半屏由 BitBLT 填写，每完成一幅就向右路基址提交一次。帧率不在 C 语言中计算。apb 子文件夹的 apb_command_dispatcher.v 以 100 MHz 计时，左路正在显示的基址相对上一拍发生变化，CPU 计数加一，右路变化则 FPGA 计数加一。计满 100000000 拍，即 1 秒后，两个计数以 BCD 锁存并清零，最大显示 99。display_engine 子文件夹下 2_layer_generators 子文件夹的 video_dual_hud.v 将 CPU 的 BCD 画在左半屏，将 FPGA 的 BCD 画在右半屏。因此两边的数字是各自半屏在上一秒内实际换页的次数。左半屏一幅画面要分成多次绘制，所以读数低于右半屏；右半屏的绘制安排在等待扫描换页的空隙中，目标是每个显示场提交一次。

压力测试关卡使用同一个计数器，仍由 apb 子文件夹的 apb_command_dispatcher.v 按扫描基址的变化计数。app 子文件夹的 main.c 在该关卡用一次提交同时写左右两个下一帧基址，两路计数因此相同。关卡画面读

取该寄存器的高字节，也就是右路计数，并画成两位数字。只有场消隐时新基址被扫描端采纳，这一秒的计数才会增加；逻辑帧如果错过消隐，显示帧率就低于75。固件另外用 100 MHz 除以 75 推进游戏时间，那是逻辑步长，不是画面上的帧率。

SPRITES 由 app 子文件夹的 main.c 在每帧测量中相加得到。相加的对象是仍存活的敌机、玩家弹、敌弹、掉落物、存活的玩家、仍有生命值的 BOSS，以及 BOSS 战中的扫描线光束。压力关的敌弹直接使用活动计数，普通关则遍历弹池。光束按描述符计数：六路符卡光束中寿命尚未结束的各计一个，预警或正在发射的切线各计一个，旧激光柱在没有切线实射覆盖时按柱数计入。一个对象只计一次，不把亮芯、彩边和光晕拆开，也不累计已经离场的弹。字段序号定义在 driver 子文件夹的 memory_map.h。压力关画面每 16 个逻辑帧采样一次这个和，DROP 峰值升高时立即再采样，避免每出生一发就擦写数字。旁边的 LOAD 是补弹目标，不是 SPRITES。

DROP 在默认编译下不由软件遍历弹池产生。dma 子文件夹的 dma_bucket_coprocessor.v 在某个 8 行组已经写满 1024 条时，把这一次登记计为一个 DROP，该弹仍留在弹池中。dma 子文件夹的 dma_scanline_prefetch.v 在可见扫描线处发现该组桶表尚未准备好时按扫描线加一，并在每场开始时清零。apb 子文件夹的 apb_command_dispatcher.v 将这两项相加，结果超过 65535 时饱和。app 子文件夹的 main.c 在协处理器结束后读取这个和；如果作业没有正常完成，则记为 65535。画面上的 DROP 不是本帧数值，而是当前 LOAD 以来的峰值，改变 LOAD 时该峰值清零。若编译时打开软件对照分桶，则由同一文件在组满时自行累加；默认编译不使用这条路径。

设置菜单的硬件加速项由 app 子文件夹的 main.c 切换。打开时，矩形填充、拷贝、Alpha 混合和饱和加法交给 BitBLT；关闭后，同样的调用改为 CPU 逐像素读写 DDR，颜色键和 Alpha 也由 CPU 计算。弹幕扫描、激光和泛光仍由显示硬件完成，不随该开关关闭。帧率的计算方式不变，仍然是上一秒内扫描基址发生变化的次数。CPU 逐像素绘图更容易错过场消隐，所以同一计数器的读数下降。对比的是两种绘图路径下的换页次数，而不是另一套帧率公式。

部分代码：

apb_command_dispatcher.v

```
module apb_command_dispatcher #(
    parameter CLOCK_HZ = 100_000_000

    wire [16:0] bucket_drop_sum={1'b0,bucket_drop_count}+{5'd0,bucket_pool_drop};
    wire [15:0] bucket_drop_word=bucket_drop_sum[16] ? 16'hffff : bucket_drop_sum[15:0];

    wire left_swap  = (vdma_active_base != left_previous);
    wire right_swap = (vdma_right_active_base != right_previous);

    // Command validation
    wire pixel_aligned = (cmd_src_addr[1:0] == 2'd0) && (cmd_dst_addr[1:0] == 2'd0);
    wire needs_pixel_align = (cmd_op_type == 2'd2) || (cmd_op_type == 2'd3) ||
                          (cmd_colorkey_en && (cmd_op_type == 2'd1));

    wire cfg_bad = (cmd_rect_w != 11'd0) && (cmd_rect_h != 11'd0) &&
                  ((cmd_rect_w > 11'd1366) ||
                   (cmd_rect_h > 11'd768) ||
                   (cmd_dst_stride[3:0] != 4'd0) ||
                   ((cmd_op_type != 2'd0) && (cmd_src_stride[3:0] != 4'd0)) ||
                   ((cmd_op_type == 2'd2) && !cmd_alpha_en) ||
                   (needs_pixel_align && !pixel_aligned));

    function [7:0] increment_bcd;
        input [7:0] value;
        begin
            if (value == 8'h99) increment_bcd = value;
            else if (value[3:0] == 9) increment_bcd = {value[7:4] + 4'd1, 4'd0};
            else increment_bcd = value + 8'd1;
        end
    endfunction

    // Count presented half-frames & compute BCD FPS
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            second_ticks   <= 32'd0;
            cpu_frames     <= 32'd0;
            fpga_frames    <= 32'd0;
            cpu_count      <= 8'd0;
            fpga_count     <= 8'd0;
            cpu_fps_bcd    <= 8'd0;
            fpga_fps_bcd   <= 8'd0;
            left_previous  <= 28'h1000000;
```

```
            right_previous <= 28'h1000000;
        end else begin
            left_previous  <= vdma_active_base;
            right_previous <= vdma_right_active_base;
            if (left_swap)  cpu_frames  <= cpu_frames + 1'b1;
            if (right_swap) fpga_frames <= fpga_frames + 1'b1;
            if (second_ticks == CLOCK_HZ - 1) begin
                second_ticks <= 32'd0;
                cpu_fps_bcd  <= left_swap ? increment_bcd(cpu_count) : cpu_count;
                fpga_fps_bcd <= right_swap ? increment_bcd(fpga_count) : fpga_count;
                cpu_count    <= 8'd0;
                fpga_count   <= 8'd0;
            end else begin
                second_ticks <= second_ticks + 1'b1;
                if (left_swap)  cpu_count  <= increment_bcd(cpu_count);
                if (right_swap) fpga_count <= increment_bcd(fpga_count);
            end
        end

            // One APB transaction publishes a full frame. Independent 0x20/0x40
            // writes remain available for the split-screen renderer benchmark.
            if (apb_write && apb_paddr == 16'h0134) begin
                vdma_next_base      <= apb_pwdata[27:0];
                vdma_right_next_base <= apb_pwdata[27:0];
            end

                8'h20: apb_prdata = {(bucket_busy | bucket_start), bucket_done, 1'b1, bucket_fault,
12'd0, bucket_drop_word};
```

video_dual_hud.v

```
`timescale 1ns/1ps
// 2x 8x12 scan overlay: CPU: NN FPS (yellow), FPGA: NN FPS (green).
// FPS inputs are frame-stable packed BCD values, measured from actual swaps.
// Split and window bounds follow the 1280x720 scan geometry: the half-screen
// boundary is x=640 and the right overlay starts at 656 (= 640 + 16).
module video_dual_hud (
 input wire [11:0] x,
 input wire [10:0] y,
 input wire [7:0] cpu_fps_bcd, fpga_fps_bcd,
 output wire visible,
 output wire [23:0] rgb
);
wire right_half = x >= 684;
wire [11:0] local_x = x - (right_half ? 12'd700 : 12'd16);
```

```
wire [10:0] local_y = y - 11'd8;
wire [3:0] cell_index = local_x[7:4];
wire [3:0] row = local_y[4:1];
wire [2:0] col = local_x[3:1];
wire [7:0] fps = right_half ? fpga_fps_bcd : cpu_fps_bcd;
assign visible = (y>=8 && y<32) && (right_half ? (x>=700 && x<892) : (x>=16 && x<192));
reg [4:0] character;
always @* begin
 character=14;
 if(right_half) begin
  case(cell_index)
   0: character=10; 1: character=11; 2: character=17; 3: character=18;
   4: character=13; 6: character={1'b0,fps[7:4]}; 7: character={1'b0,fps[3:0]};
   9: character=10; 10: character=11; 11: character=12;
   default: character=14;
  endcase
 end else begin
  case(cell_index)
   0: character=15; 1: character=11; 2: character=16; 3: character=13;
   5: character={1'b0,fps[7:4]}; 6: character={1'b0,fps[3:0]};
   8: character=10; 9: character=11; 10: character=12;
   default: character=14;
  endcase
 end
end
reg [95:0] bitmap;
always @* begin
 case(character)
  5'd0: bitmap=96'h3c66c3c3c3c3c3c3c3663c;
  5'd1: bitmap=96'h183878181818181818187e;
  5'd2: bitmap=96'h3c66c303060c183060c0c3ff;
  5'd3: bitmap=96'h3c66c303061c060303c3663c;
  5'd4: bitmap=96'h060e1e3666c6ff060606060f;
  5'd5: bitmap=96'hffc0c0c0fc06030303c3663c;
  5'd6: bitmap=96'h3c66c0c0fcc6c3c3c3c3663c;
  5'd7: bitmap=96'hffc306060c0c181830303030;
  5'd8: bitmap=96'h3c66c3c3663c66c3c3c3663c;
  5'd9: bitmap=96'h3c66c3c3c3c33f030303663c;
  5'd10: bitmap=96'hffc0c0c0fcc0c0c0c0c0c0c0;
  5'd11: bitmap=96'hfcc6c3c3c3c6fcc0c0c0c0c0;
  5'd12: bitmap=96'h3c66c3c0c0780e0303c3663c;
  5'd13: bitmap=96'h000018180000001818000000;
  5'd14: bitmap=96'h000000000000000000000000;
  5'd15: bitmap=96'h3c66c3c0c0c0c0c0c0c3663c;
```

```
  5'd16: bitmap=96'hc3c3c3c3c3c3c3c3c3c3663c;
  5'd17: bitmap=96'h3c66c3c0c0cfc3c3c3663e;
  5'd18: bitmap=96'h183c66c3c3c3ffc3c3c3c3;
  default: bitmap=0;
 endcase
end
wire [6:0] bit_index = 7'd95 - {row,3'b000} - {4'd0,col};
wire foreground = bitmap[bit_index];
assign rgb = foreground ? (right_half ? 24'h00ff00 : 24'hffff00) : (right_half ? 24'h0000ff :
24'h000000);
endmodule
```

main.c

```
/* One small unit of the left half.  Also starts and finishes a left frame. */
static void cpu_left_step(uint32_t *left_back)
{
    if (!cpu_running) {
        cpu_fb      = *left_back;
        cpu_phase   = 0;
        cpu_y       = 0;
        cpu_box     = 0;
        cpu_running = 1;
    }

    if (cpu_phase < 2) {
        int half = (int)(SCREEN_HEIGHT / 2u);
        int n    = half - cpu_y;
        if (n > CPU_UNIT_ROWS) n = CPU_UNIT_ROWS;
        cpu_software_fill(cpu_fb, 0, cpu_phase * half + cpu_y,
                          (int)SCAN_HALF_PIXELS, n, 0x00000000u);
        cpu_y += n;
        if (cpu_y >= half) { cpu_phase++; cpu_y = 0; }
        return;
    }

    {
        int end = cpu_box + CPU_UNIT_BOXES;
        for (int i = cpu_box; i < end && i < NUM_ENTITIES; i++) {
            int w = (int)entities[i].w;
            int h = (int)entities[i].h;
            int x = (int)entities[i].x;
            int y = (int)entities[i].y;
            if (y >= (int)SCREEN_HEIGHT || y + h <= 0) continue;
            cpu_software_fill(cpu_fb, x, y, w, h, entities[i].color);
```

```
        }
        cpu_box = end;
    }

    if (cpu_box >= NUM_ENTITIES) {
        gpu2d_flush_dcache();
        GPU2D_REG_LEFT_NEXT = cpu_fb;
        *left_back ^= 0x00600000u;
        cpu_running = 0;
    }

                if (want >= 0) {
                    screen_state = want;
                    board_set_hud_disable(screen_state != SCREEN_BITBLT);

        } else if (screen_state == SCREEN_BITBLT) {
            /* The left half's software pass runs inside this wait - see
             * cpu_left_step() for why it is here and not after the BitBlt
             * pass, which is what made the right half miss every second
             * display frame. */
            uint32_t hs_guard = 2000000u;
            while (GPU2D_REG_RIGHT_ACTIVE == right_back && --hs_guard) {
                cpu_left_step(&left_back);
            }
            if (!hs_guard) continue;

            /* ---- FPGA accelerated right half ------------------------- */
            gpu2d_fill(right_back + SCAN_HALF_BYTES, SCAN_RIGHT_PIXELS, SCREEN_HEIGHT, 0x00000000u,
SCREEN_STRIDE);

            for (int i = 0; i < NUM_ENTITIES; i++) {
                int fx = ((int)entities[i].x * (int)SCAN_RIGHT_PIXELS) / (int)SCAN_HALF_PIXELS;
                uint32_t dst = right_back + SCAN_HALF_BYTES +
                               (uint32_t)(entities[i].y * SCREEN_STRIDE) +
                               (uint32_t)(fx * 4);
                gpu2d_fill(dst, (uint32_t)entities[i].w, (uint32_t)entities[i].h,
                           entities[i].color, SCREEN_STRIDE);
            }

            gpu2d_blit_sprite(right_back + SCAN_HALF_BYTES + ((SCREEN_HEIGHT - 120u) *
SCREEN_STRIDE) +
                              ((SCAN_RIGHT_PIXELS - 120u) * 4u),
                              MM_BOSS_SPRITE, MM_BOSS_SPRITE_W, MM_BOSS_SPRITE_H,
                              MM_BOSS_SPRITE_STRIDE, SCREEN_STRIDE);
```

```
            gpu2d_wait_idle();
            GPU2D_REG_RIGHT_NEXT = right_back;
            right_back ^= 0x00600000u;


            telemetry_write();
            if (gpu2d_stalled) continue;
            GPU2D_REG_FRAME_NEXT = right_back;
            right_back ^= 0x00600000u;

/* ---- fixed-step game time ------------------------------------------------
 * The playable stage simulates on FIXED 1/75 s ticks driven by the board's
 * core-tick counter, so a dropped display frame skips a picture, never game
 * time.  LOGIC_STEP_CYCLES is exactly one 75 Hz frame at 100 MHz.  The
 * accumulator lives in main(); LOGIC_MAX_STEPS caps the catch-up burst so a
 * pathologically slow frame cannot spiral into an even slower one. */
#define LOGIC_HZ          75u
#define LOGIC_STEP_CYCLES  (100000000u / LOGIC_HZ)   /* 1333333 */
#define LOGIC_MAX_STEPS    4u

    draw_text(fb,HUD_BACKEND_X,HUD_BACKEND_Y,hw_accel ? "FPGA" : "CPU",2,
              hw_accel ? 0x0070F0CFu : 0x00FFC070u);
    {
        uint32_t bcd = (GPU2D_REG_FPS_BCD >> 8) & 0xFFu;
        char digits[3] = { (char)('0'+((bcd >> 4)&15u)), (char)('0'+(bcd&15u)), 0 };
        /* FPS shares the FPGA/CPU row.  At its old spot (y=534) it was painted
         * over the backdrop's RENDER BACKEND label, and at y=556 it collided
         * with the FPGA token itself. */
        hud_label(fb, HUD_LBL_FPS);
        draw_text(fb,HUD_FPS_NUM_X,HUD_FPS_NUM_Y,digits,2,0x0070F0CFu);

static void telemetry_write(void)
{
    int ne = 0, np = 0, nb = 0, ni = 0;
    for (int i = 0; i < MAX_ENEMIES; i++)  if (enemies[i].flags  & EF_ACTIVE) ne++;
    bullet_t *b;

    FOR_EACH_PBULLET(b) np++;
    if (is_stress_stage) nb = active_ebullets_count;
    else { FOR_EACH_EBULLET(b) nb++; }
    for (int i = 0; i < MAX_ITEMS; i++)    if (items[i].flags    & EF_ACTIVE) ni++;

    tlm[TLM_MAGIC]    = TLM_MAGIC_VAL;
```

```
tlm[TLM_PLAYER_X] = (uint32_t)player_x;
tlm[TLM_PLAYER_Y] = (uint32_t)player_y;
tlm[TLM_LIVES]    = lives;
tlm[TLM_SCORE]    = score;
tlm[TLM_WEAPON]   = weapon;
tlm[TLM_ATK]     = atk;
tlm[TLM_FIRE_CD]  = fire_cd;
tlm[TLM_KILLS]    = kills;
tlm[TLM_ENEMIES]  = (uint32_t)ne;
tlm[TLM_PBULLETS] = (uint32_t)np;
tlm[TLM_EBULLETS] = (uint32_t)nb;
tlm[TLM_ITEMS]    = (uint32_t)ni;
/* Every pooled shot is in nb, including Q4 spells and reserved orbs.
 * Scanout beams have no pool record: count each warning/live beam once. */
unsigned beams = 0;
if (boss_state == BOSS_FIGHT) {
    unsigned bars = 0;
    for (unsigned i = 0; i < 6; i++) beams += wave > 0 && spell->beams[i].ttl != 0;
    for (unsigned i = 0; i < CUT_SLOTS; i++) {
        beams += beam_phase[i] != 0;
        bars |= beam_phase[i] == 2;
    }
    if (boss_laser && !bars) beams += BOSS_LASER_COLS;
}
tlm[TLM_BOSS_BEAMS] = beams;
tlm[TLM_SPRITES]  = (uint32_t)(ne + np + nb + ni) + beams + (lives != 0) +
                    (boss_state != BOSS_NONE && boss_hp > 0);

    /* The live FPS readout.  It is a hardware counter, so a locked display
     * leaves it alone - which is exactly the steady state this ledger is
     * for - but a frame that slipped still has to erase its old digits. */
    hud_watch(HUD_SIG_FPS, (GPU2D_REG_FPS_BCD >> 8) & 0xFFu,
              HUD_BAND_FPSNUM);
    /* Signature: [11:0] population, [31:16] drop peak.
     * Y explicitly invalidates this watcher when the load changes.
     * Sample the last published population. Hold it for 16 logic ticks
     * so the HUD does not add glyph blits on every projectile birth.
     * Both buffers still receive each sampled value via the dirty ledger. */
    if (is_stress_stage && ((frame_tick & 15u) == 0u ||
                            hud_sig[HUD_SIG_STRESS] == 0u ||
                            bt_drop_peak > (hud_sig[HUD_SIG_STRESS] >> 16)))
        hud_watch(HUD_SIG_STRESS,
                  (tlm[TLM_SPRITES] & 65535u) |
                  (bt_drop_peak << 16), HUD_BAND_STRESS);
```

```
        if (is_stress_stage && (band & HUD_BAND_STRESS)) {
            HUD_CLEAR_L(592u, 112u);
            draw_text(fb, HUD_SPRITES_TAG_X, HUD_SPRITES_TAG_Y,
                      "SPRITES", 2, 0x0070F0CFu);
            draw_number(fb, HUD_SPRITES_NUM_X, HUD_SPRITES_NUM_Y,
                        hud_sig[HUD_SIG_STRESS] & 65535u, 5, 0x0070F0CFu);
            draw_text(fb, HUD_LOAD_TAG_X, HUD_LOAD_TAG_Y,
                      "LOAD", 2, 0x0070F0CFu);
            draw_number(fb, HUD_LOAD_NUM_X, HUD_LOAD_NUM_Y,
                        stress_target, 5, 0x0070F0CFu);
            draw_text(fb, HUD_DROPS_TAG_X, HUD_DROPS_TAG_Y,
                      "DROP", 2, 0x00FFC070u);
            draw_number(fb, HUD_DROPS_NUM_X, HUD_DROPS_NUM_Y,
                        hud_sig[HUD_SIG_STRESS] >> 16, 5, 0x00FFC070u);
        }

static void __attribute__((used, retain, noinline)) bucket_build_finish(void)
{
    uint32_t status;
    do {
        status = GPU2D_REG_BUCKET_ACCEL;
        if (profile_elapsed(bt_started, GPU2D_REG_CORE_TICKS) > 8000000u) break;
    } while (status & 0x80000000u);
    if ((status & 0xB0000000u) != 0x20000000u) {
        /* Fail visibly instead of publishing a partial table as DROP=0. */
        GPU2D_REG_BUCKET_BASE = 0u;
        bt_dropped = 65535u;
    } else {
        /* DMA owns the enemy records until DONE; discard clean CPU reads
         * made by lance rendering before consuming the DMA writeback. */
        gpu2d_flush_dcache();
        if (eb_motion_job) stress_coprocessor_finish();
        GPU2D_REG_BUCKET_BASE = MM_BUCKET_TABLE;
        bt_dropped = status & 0xFFFFu;
        const volatile uint16_t *used = (const volatile uint16_t *)MM_BUCKET_USED
                                      + bt_buf * MM_BUCKET_ROWS;
        for (unsigned g = 0; g < MM_BUCKET_ROWS; ++g) bt_count[g] = used[g];
    }
    if (bt_dropped > bt_drop_peak) bt_drop_peak = bt_dropped;
    if (bt_dropped > bt_drop_window) bt_drop_window = bt_dropped;
    tlm[TLM_BUCKET_DROPS] = bt_dropped;
}
```

```
    if (confirm_edge) {
        switch (settings_sel) {
        case 0: infinite_respawn ^= 1; break;
        case 1: hw_accel ^= 1;        break;

static void cpu_blit(uint32_t dst, uint32_t src, int w, int h,
                     int sstride, int dstride)
{
    for (int y = 0; y < h; y++) {
        volatile uint32_t *drow = (volatile uint32_t *)(dst + (uint32_t)(y * dstride));
        const volatile uint32_t *srow = (const volatile uint32_t *)(src + (uint32_t)(y * sstride));
        for (int x = 0; x < w; x++) {
            uint32_t px = srow[x];
            if (px != MAGENTA_KEY) drow[x] = px;      /* colour key, in software */
        }
    }
}

static void cpu_alpha(uint32_t dst, uint32_t src, int w, int h,
                      int sstride, int dstride, uint32_t ga)
{
    for (int y = 0; y < h; y++) {
        volatile uint32_t *drow = (volatile uint32_t *)(dst + (uint32_t)(y * dstride));
        const volatile uint32_t *srow = (const volatile uint32_t *)(src + (uint32_t)(y * sstride));
        for (int x = 0; x < w; x++) {
            uint32_t s = srow[x];
            uint32_t d = drow[x];
            if (s == MAGENTA_KEY) continue;          /* colour key beats alpha */
            uint32_t a = (((s >> 24) & 0xFFu) * ga + 127u) / 255u;
            uint32_t out = d & 0xFF000000u;          /* background alpha kept  */
            for (int sh = 0; sh <= 16; sh += 8) {
                uint32_t sc = (s >> sh) & 0xFFu;
                uint32_t dc = (d >> sh) & 0xFFu;
                uint32_t oc = (sc * a + dc * (255u - a) + 127u) / 255u;
                out |= oc << sh;
            }
            drow[x] = out;
        }
    }
}

static void cpu_fill(uint32_t dst, uint32_t w, uint32_t h, uint32_t color,
                     uint32_t stride)
{
```

```
    for (uint32_t y = 0; y < h; y++) {
        volatile uint32_t *row = (volatile uint32_t *)(dst + y * stride);
        for (uint32_t x = 0; x < w; x++)
            row[x] = color;
    }
}

/* Saturating add, the software twin of OP_ADD and of the scanout's laser
 * columns: min(255, dst + src) per channel.  A colour-keyed source pixel is
 * skipped - same rule as cpu_blit() and cpu_alpha(), and the same rule the
 * engine applies before it blends. */
static void cpu_add(uint32_t dst, uint32_t src, int w, int h,
                    int sstride, int dstride)
{
    for (int y = 0; y < h; y++) {
        volatile uint32_t *drow = (volatile uint32_t *)(dst + (uint32_t)(y * dstride));
        const volatile uint32_t *srow = (const volatile uint32_t *)(src + (uint32_t)(y * sstride));
        for (int x = 0; x < w; x++) {
            uint32_t s = srow[x];
            if (s == MAGENTA_KEY) continue;
            uint32_t d = drow[x];
            uint32_t out = d & 0xFF000000u;          /* background alpha kept */
            for (int sh = 0; sh <= 16; sh += 8) {
                uint32_t sum = ((s >> sh) & 0xFFu) + ((d >> sh) & 0xFFu);
                out |= (sum > 255u ? 255u : sum) << sh;
            }
            drow[x] = out;
        }
    }
}

static void r_fill(uint32_t dst, uint32_t w, uint32_t h, uint32_t color,
                   uint32_t stride)
{
    damage_mark(dst, w, h, stride);
    if (hw_accel) {
        blit_cmds++; blit_rows += h;
        gpu2d_fill(dst, w, h, color, stride);
    } else {
        cpu_fill(dst, w, h, color, stride);
    }
}

static void r_blit(uint32_t dst, uint32_t src, uint32_t w, uint32_t h,
```

```
                uint32_t sstride, uint32_t dstride)
{
    damage_mark(dst, w, h, dstride);
    if (hw_accel) {
        blit_cmds++; blit_rows += h;
        gpu2d_blit_sprite(dst, src, w, h, sstride, dstride);
    } else {
        cpu_blit(dst, src, (int)w, (int)h, (int)sstride, (int)dstride);
    }
}

static void r_add(uint32_t dst, uint32_t src, uint32_t w, uint32_t h,
                uint32_t sstride, uint32_t dstride)
{
    damage_mark(dst, w, h, dstride);
    if (hw_accel) {
        blit_cmds++; blit_rows += h;
        gpu2d_add(dst, src, w, h, sstride, dstride);
    } else {
        cpu_add(dst, src, (int)w, (int)h, (int)sstride, (int)dstride);
    }
}

static uint32_t r_add_saved_key;   /* colour-key state before a batch */

static void r_add_begin(void)
{
    if (!hw_accel) return;
    /* SAVE AND RESTORE, never a bare on/off pair.  The colour key is a mode of
     * the engine that other draws can also want - the pressure stage turns it
     * on when it starts and everything after that depends on it still being on.
     * A batch that simply switched it off at the end would silently disarm
     * every later blit in the frame, and a sprite whose key is off blits its
     * magenta background into the picture. */
    r_add_saved_key = GPU2D_REG_COLORKEY_CTRL & 1u;
    gpu2d_set_colorkey(1, MAGENTA_KEY);
}

static void r_add_end(void)
{
    if (hw_accel) gpu2d_set_colorkey(r_add_saved_key, MAGENTA_KEY);
}

static void r_alpha(uint32_t dst, uint32_t src, uint32_t w, uint32_t h,
```

```
                  uint32_t sstride, uint32_t dstride, uint32_t ga)
{
    damage_mark(dst, w, h, dstride);
    if (hw_accel) {
        blit_cmds++; blit_rows += h;
        gpu2d_alpha_blit(dst, src, w, h, sstride, dstride);
    } else {
        cpu_alpha(dst, src, (int)w, (int)h, (int)sstride, (int)dstride, ga);
    }
}

static void r_alpha_begin(uint32_t ga)
{
    if (hw_accel) gpu2d_set_alpha(1, ga);
}

static void r_alpha_end(void)
{
    if (hw_accel) gpu2d_set_alpha(1, 255);
}

static void r_sync(void)
{
    if (hw_accel) gpu2d_wait_idle();
}
```

dma_bucket_coprocessor.v

```
              ST_WRITE_GROUPS:begin
                  if(group_count>=TABLE_SLOTS) begin
                      if(drop_count!=16'hffff) drop_count<=drop_count+16'd1;
                      if(group_index==group_last) state<=ST_NEXT;
                      else group_index<=group_index+7'd1;
                  end else begin
```

dma_scanline_prefetch.v

```
    wire [10:0] target_y = v_cnt == V_TOTAL-1 ? 11'd0 : v_cnt+11'd1;
    assign entry_ready = valid[entry_group] && pixel_tag[entry_group] == target_y[9:3];
    integer k;
    always @(posedge pixel_clk or negedge pixel_rst_n) begin
        if (!pixel_rst_n) begin
            pub_s1 <= 0; pub_s2 <= 0; pub_seen <= 0; valid <= 0;
            pixel_tag[0] <= 0; pixel_tag[1] <= 0; missed_lines <= 0;
        end else begin
```

```
            pub_s1 <= published; pub_s2 <= pub_s1; pub_seen <= pub_s2;
            for (k=0;k<2;k=k+1) if (pub_s2[k] != pub_seen[k]) begin
                pixel_tag[k] <= tags[k]; valid[k] <= 1;
            end
            if (h_cnt == 0 && v_cnt == 768) valid <= 0;
            if (h_cnt == 0 && v_cnt == 769) missed_lines <= 0;
            else if (h_cnt == 64 && target_y < 768 && !entry_ready &&
                    missed_lines != 4095) missed_lines <= missed_lines+1'b1;
        end
    end
```

memory_map.h

```
#define TLM_SPRITES     57u   /* actors, items, pooled shots and BOSS beams */
#define TLM_BUCKET_DROPS 58u  /* rejected scanline entries, not whole sprites */
#define TLM_BOSS_BEAMS   59u  /* warning/live scanout beams, one per descriptor */
```

1.6 主要创新点

1.控制、搬运与显示独立的异构加速体系：针对 CPU 逐像素绘图开销大的瓶颈，构建了 RISC-V 软核与 FPGA 协同的“控制分发—BitBLT 光栅化—DMA 提取—流水显示”架构。CPU 仅经 APB3 下发图元参数；BitBLT 引擎通过 128 位 AXI4 总线自主执行填充、拷贝、颜色键抠图、Alpha 混合与饱和加法。其内部采用三路并行地址生成器实现读写突发重叠，结合桶形移位器解决非对齐贴图错位，并以移位公式在分子不超过 65152 时精确完成除以 255。每个 128 位拍含 4 个像素，每个像素的 R、G、B 各做一次有符号 9×8 乘法，使复杂场景帧率由纯软件的 25 FPS 提升至稳定 75 FPS。

2. 零显存写入弹幕与扫描线行缓存预存储：为突破万级弹幕逐帧读写显存的数 GB/s 带宽墙，提出弹幕不入帧缓冲的扫描线实时合成机制。DDR3 仅保存 8 字节紧凑状态与按 8 行划分的 96 组空间行桶表，片上 BRAM 将 5 位形状点阵与 3 位调色板色相正交。扫描阶段采用双组缓存乒乓预取（整帧桶表读取仅约 29.5 MB/s），行渲染器利用 32 个交错 Bank 并行行缓存在单像素时钟内无冲突展开整行最多 26 个精灵像素，实现 20000 发弹幕对帧缓冲的零字节写入与零背景修补开销。

3. AXI 突发硬件协调处理器：针对海量弹幕下 CPU 遍历更新与软件构桶的算力瓶颈，设计了硬件分桶协处理器。该模块将敌弹位移积分、越界回收、31×31 玩家危险框碰撞候选筛选、空闲槽索引生成、朝向图块映射及跨组行桶登记融为一体；通过复用块捕获寄存器将 32 颗弹幕合并为 16 拍 128 位 AXI4 突发读写，并依据每组历史水位自动清零双缓冲旧桶尾部。

4.多图层合成流水线与 SRL8/DSP 实时后处理：针对大面积动态特效，显示引擎在纯净的 85.714 MHz 像素流上构建了多图层实时生成与四级混合管线，自下而上依次为天气与弹幕、激光、冲击环与符卡光束、自机光束与 HUD。硬件按像

素坐标实时运算 170 点 xorshift32 天气粒子、4 路高斯等离子激光、自机光束与预警线、程序化冲击环、基于 Q8 向量外积的 6 路任意角度光束及 OSD 图像。后处理利用 SRL8 移位寄存器与行缓存构建各向同性卷积核及高斯泛光（Bloom），并串联 9 个 DSP 组成的 3×3 仿射色彩矩阵，在不占 CPU 周期与 DDR3 带宽的前提下实现丰富光影特效。

5.防欠载机制与跨时钟域防画面撕裂机制：为保障 1366×768@75Hz 扫描线（有效行约 15.94 μs，含消隐的整行约 16.99 μs）在总线争用下数据不间断、画面不撕裂，设计了四重防撕裂机制：一是写守卫监测异步 FIFO 深度，水位偏低时暂停新写请求，已发出的写地址保持到握手完成，并在最多 256 个核心时钟后放行一次写，避免读写活锁；二是 128b-32b 转换器在读空时继续推进相位，把缺字记为迟到并在数据返回时丢弃，防止整行像素横向错位；三是将消隐信号延长至 16 个像素周期，防止跨时钟域漏采；四是采用消隐周期锁定显存地址与翻转信号握手,防止画面撕裂。

6.TCM 预加载与低带宽异构音视频协同：为消除 CPU 从 DDR3 取指的延迟，固件经 LTO 压缩，随 FPGA 位流预载实现搬运冷启动；同时将美术特效与查找表资源经 LZSS+RLE 压缩进 16 KiB 片上 ROM，开机快速解压至 DDR3。外设端由硬件状态机以 100 kHz、每秒 50 次轮询地址 0x5A 的 I2C 手柄，板载按键另做 10 ms 消抖。摇杆以中心值 128 为原点，偏离 28 才判为推动，回到距中心 14 以内才松开，方向变化要连续两帧一致，确认后只输出 40、128、216 三档，这部分在 gamepad.c。音频链将 SPI Flash 中的 12 kHz BGM 与片上 8192 字节 ROM 音效采样混合后通过 I2S 送入功放，全程不占用 DDR3 带宽。

- ## 1.7 设计流程

设计流程软硬件同时推进。先划分职责 RISC-V 运行游戏逻辑并经 APB3 提交命令，FPGA 承担像素搬运、图层合成与后处理，两者通过 128 位 AXI 共享 DDR3。RTL 按 APB 派发、BitBLT、DMA 预取、显示引擎分模块实现，固件用 C 语言与之共用同一份内存映射。代码验证分三层：仿真模拟器跑通游戏流程，Verilog 测试台判断时序等内容是否合规，Efinity 官方 IDE 工具负责综合,布线等并给出最终报告。上板后按 FPS、SPRITES、DROP 优先级逐档上升并通过串口数据判断问题所在，进而修改 RTL 或固件，直至时序,资源占用,与帧率同时达标。

第二部分　系统组成及功能说明

2.1 整体介绍

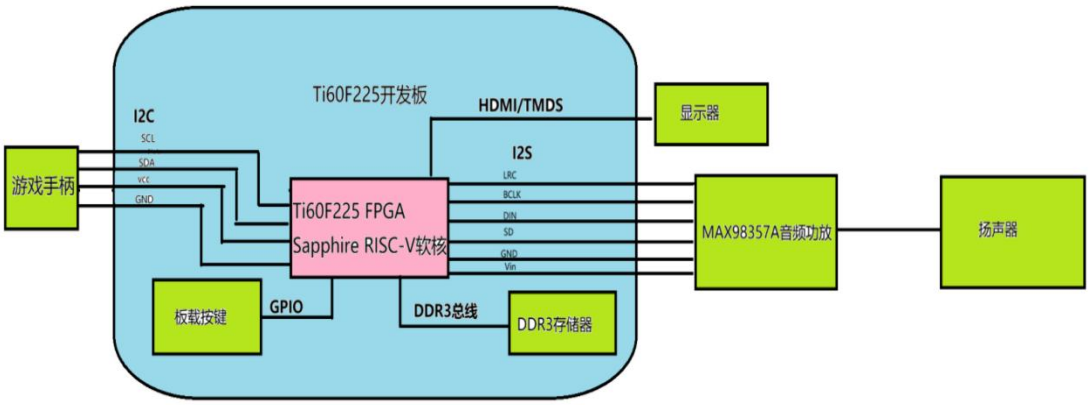


图 2-1 系统硬件组成与接口连接图

图 2-1 展示了本作品的系统硬件组成与接口连接关系。系统以易灵思 Titanium Ti60F225 FPGA 为核心，集成 Sapphire RISC-V 软核，运行游戏逻辑与图形驱动程序，并由硬件完成图像搬运、混合与显示合成。板载 DDR3 存储器通过 DDR3 总线与 FPGA 连接，用于存放图像资源、双帧缓冲及弹幕数据，为处理器和图形加速模块提供共享存储空间。

在人机交互方面，游戏手柄通过 I2C 协议与 FPGA 通信，连接至 J7 转接板；图像经 DVI 编码后，通过 HDMI 接口输出至显示器。音频模块通过 I2S 接口连接 MAX98357A 芯片数字音频功放，驱动扬声器播放背景音乐与游戏音效。此外，系统通过 UART/JTAG 接口与 PC 调试端连接，用于查看信息、下载代码与调试，形成集交互、图形处理、音视频输出和开发调试于一体的嵌入式图形演示平台。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件平台是易灵思 Titanium 系列的 Ti60F225I3 开发板（Ti60F225-DK）。核心器件 Ti60F225 采用 16 nm 工艺、60K 逻辑单元、225 球 FBGA 封装，片内集成 4 个 PLL、160 个 DSP 和 256 个内存块。开发板提供 25 MHz 差分晶振、一片 16 位 DDR3（MT41J128M16JT-125，800 Mbps）、HDMI 接口、40 针扩展口(J7 LVDS 差分)、4 个 LED 和 2 个按键，板载 FT4232HL，一根 USB 线即可完成供电、JTAG 下载与 UART 通信。

系统划分为四个部分。计算与存储由 Sapphire RISC-V 软核和 256 MB DDR3 组成，软核跑游戏逻辑与驱动，DDR3 存放帧缓冲、关卡背景、精灵图库、弹幕池与行桶表。图形通路由 BitBLT 引擎、行桶协处理器、扫描线合成层和后处理滤镜组成。显示输出由 DVI 编码器和 TMDS 差分对经 HDMI 送显示器。外设包括 I2C 手柄、I2S 功放、按键和调试串口。连接关系见图 2-1。

FPGA 的 IO 电压按功能分 bank：DDR3 占 3B、4A、4B，固定 1.5 V；HDMI 用 3A bank，1.8 V；I2C、I2S、按键与串口位于 3.3 V 的 TL 与 TR 等 bank。核心电压 0.95 V，PLL 模拟电源单独供电。开发板实物如图 2-2 所示。

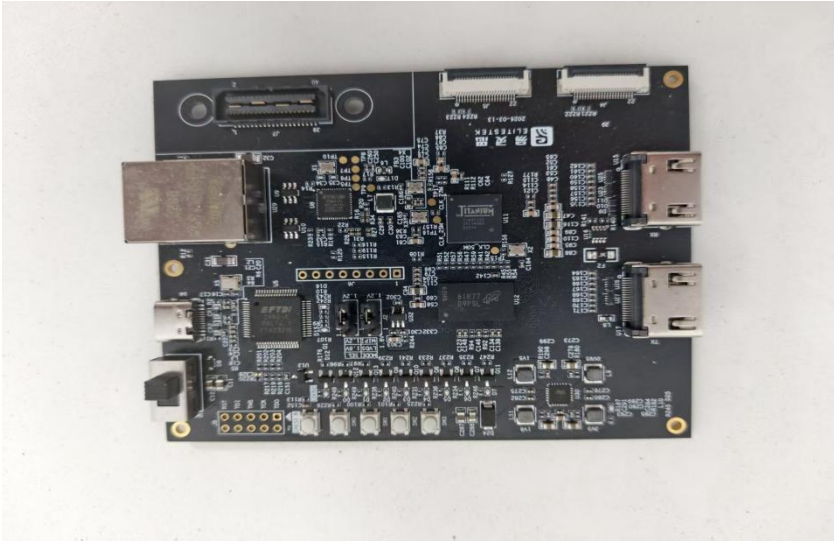


图 2-2 开发板实物图

2.2.2 电路各模块介绍

电路按信号走向分八块：电源与复位、时钟与 PLL、DDR3 存储、HDMI 输出、I2C 手柄接口、音频输出、按键与调试接口、40 针扩展口。各块的关键信号与连接如下。

（1）电源与复位。板上有 0.95 V 核心电源、PLL 模拟电源、辅助电源和三组 IO 电源；稳定后经 tCRESET_N 释放配置复位。关键信号是三路锁定指示 pll_locked、user_pll_locked、hdmi_tx_locked，以及 sys_rst 与 ddr_rstn。系统复位需三个 PLL 全部锁定且 DDR3 校准完成（cal_done），否则软核停在复位态。

（2）时钟与 PLL。25 MHz 差分晶振从 clk_25m（GPIOL_P_08）进入，分三路。PLL_TL0 产生 core_clk 100 MHz，供软核、AXI 总线、BitBLT 与 DDR

本地总线。PLL_TR0 产生 sdram_clk、rx_cal_clk、tx_cal_clk、tx_cal_clk_90edge 四路 400 MHz 时钟，供 DDR3 物理层。PLL_BR0 产生 hdmi_tx_slow_clk 85.714 MHz（像素时钟）和 hdmi_tx_fast_clk 428.57 MHz（TMDS 5 倍串行时钟）。

（3）DDR3 存储电路。板载 MT41J128M16JT-125，16 位数据线，8 个 bank，14 位行地址、10 位列地址，最高 800 Mbps，容量 256 MB，接在 3B、4A、4B 三个 bank 上，1.5 V SSTL。关键信号含 ddr_ck、ddr_cs_n、ddr_ras_n / ddr_cas_n / ddr_we_n、ddr_ba、ddr_addr、ddr_dq、ddr_dqs、ddr_dm 与 ddr_odt。上电后完成 DQS 门控与校准，cal_done 拉高才允许访问。

（4）HDMI 输出电路。HDMI 由 FPGA 高速 IO 以 LVDS 差分对实现，占 3A bank，1.8 V。四对差分信号为时钟 tmds_clk_p/n 与三路数据 tmds_data0~2 的正负端。本作品只做发送：8 位像素经 encode.v 编成 10 位码字，再由 5:1 串化器在 428.57 MHz 送出。

（5）I2C 手柄接口。手柄接 J7.37（SCL）与 J7.39（SDA），3.3 V 弱上拉，FPGA 侧开漏输出，速率 100 kHz。器件为 emakefun Mini-Joystick，地址 0x5A，内部用 CH32V003F4U6 采样摇杆与 A、B、C、D、OK 五个按键，对外只引出 G、V、SCL、SDA。硬件按固定周期轮询，读回 joy_x、joy_y（0～255）与 5 位按键。见图 2-3 至图 2-5。

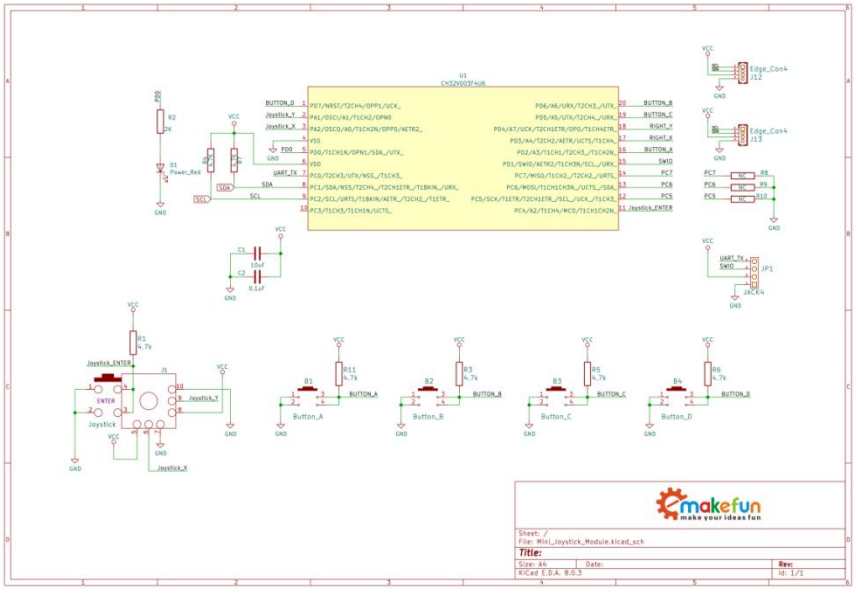


图 2-3 手柄模块原理图



图 2-4 手柄模块实物

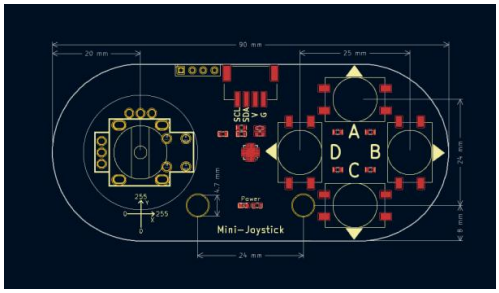


图 2-5 手柄图纸

（6）音频输出电路。I2S 三根信号从 J7 引出：audio_i2s_bclk 在 A12（J7.40），3.125 MHz；audio_i2s_lrclk 在 A4（J7.34），48.828125 kHz；audio_i2s_din 在 B5（J7.38），均为 3.3 V LVCMOS。外部功放为 MAX98357A，VIN 接 5 V、SD 接 3.3 V 使能、SPK± 接扬声器。背景音乐是 Flash 中的单声道 12 kHz PCM 流，经 FIFO 送 I2S；音效取自片上 8192 字节 ROM 采样，由 game_audio_synth 与 12 kHz BGM 混合。

（7）按键与调试接口。板载 KEY0（C4）与 KEY2（N14）经 i2c_gamepad_top 做 10 ms 消抖。两者按下后都回到主菜单，不切换 HUD，也不改变弹幕档位，操作见 1.2。UART 的 RX/TX 在 GPIOL_02 与 GPIOR_28；JTAG 四线经板载 FT4232HL 与 PC 相连。

（8）40 针扩展口。J7 除 5 V 与 3.3 V 电源外还引出 GPIO 与多路复用引脚，本作品只用了五根：I2S 三根（J7.34、J7.38、J7.40）与 I2C 两根（J7.37、J7.39），完整分配见图 2-6。接线要避开已被 PLL 时钟输入占用的引脚，例如 C5 与 J7.32 是 GPIOL_11_PLLIN2，不能当普通输出。

6.2 扩展口 J7 引脚分配

CONN Num.	Net Name	FPGA Pin Name	FPGA pin Num.	Bank 电压
01	VCC_3V3			
02	LVDS	GPIOL_P_14	F1	1.8V
03	VCC_5V			
04		GPIOL_N_14	F2	1.8V
05	GND			
06	GND			
07	MIPI_D0_P	GPIOT_P_00_PLLIN1	E6	1.8V/1.2V
08		GPIOL_P_13	F3	1.8V
09	MIPI_D0_N	GPIOT_N_00	D5	1.8V/1.2V
10	GPIOL_N_13	GPIOL_03	E3	1.8V
11	GND			
12	GND			
13	MIPI_D1_P	GPIOT_P_01	B6	1.8V/1.2V
14	LVDS_P4	GPIOL_P_16	D1	1.8V
15	MIPI_D1_N	GPIOT_N_01	C6	1.8V/1.2V
16	LVDS_N4	GPIOL_N_16	C1	1.8V
17	GND			
18	GND			
19	MIPI_CK_P	GPIOT_P_03	A5	1.8V/1.2V
20	LVDS_P3	GPIOL_P_17	B1	1.8V
21	MIPI_CK_N	GPIOT_N_03	A6	1.8V/1.2V
22	LVDS_N3	GPIOL_N_17	C2	1.8V
23	GND			
24	GND			
25	MIPI_D3_P	GPIOT_P_05	E7	1.8V/1.2V
26	LVDS_P2	GPIOL_P_18	B2	1.8V
27	MIPI_D3_N	GPIOT_N_05	D7	1.8V/1.2V
28	LVDS_N2	GPIOL_N_18	A2	1.8V
29	GND			
30	GND			
31	MIPI_D2_P	GPIOT_P_04	F7	1.8V/1.2V
32	GPIOL_11	GPIOL_11	C5	3.3V
33	MIPI_D2_N	GPIOT_N_04	F8	1.8V/1.2V
34	GPIOL_09	GPIOL_09	A4	3.3V
35	GND			
36	GND			
37	GPIOL_04			3.3V
38	GPIOL_10			3.3V
39	GPIOL_06			3.3V
40	GPIOR_13			3.3V

图 2-6 扩展口 J7 引脚分配（摘自开发板 DEMO 说明）

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为板端固件与 PC 端工作台两部分。板端固件用 C 写成单线程裸机程序，编译后随比特流进入片上 64 KiB BRAM；PC 端工作台负责编译、综合、仿真与资产烘焙，两端共用同一份内存映射与寄存器定义。

板端程序从 crt0.S 进入 main()，完成建栈与初始化后进入主循环。每圈先调用 audio_stream_service() 补充 PCM。菜单调用 draw_menu()，压力弹幕调用 stress_frame()，两者都写 GPU2D_REG_FRAME_NEXT。CPU 与 FPGA 对比界面的右半写 GPU2D_REG_RIGHT_NEXT，左半软件画完后写 GPU2D_REG_LEFT_NEXT。关卡在未暂停时按累积时间调用 game_update()，一圈 0 到 4 次；压力关最多 1 次，硬直时为 0。随后 game_render_pass() 出图，telemetry_write() 写出 64 字遥测，再写 GPU2D_REG_FRAME_NEXT。

gpu2d_swap_buffers() 仍在驱动中，主循环不调用。细节见 2.3.2。

PC 端有四类工具。Efinity 2026.1 负责综合与时序；sw/build_firmware.py 调 RISC-V GCC 编译固件并经 elf_to_bram.py 写入 BRAM；simulator/ 是 RV32IM 虚拟机，逐帧导出 1366×768 的 BMP；tools/ 烘焙美术资源并检查 DDR 布局。

仿真工作台由 rv32i_soc_vm.py 与对照模型组成：blit_ref.py 管 BitBLT，alpha_ref.py 管 Alpha，scanout_model.py 管扫描合成，bucket_model.py 管行桶。test_runner.py 把 28 个场景分组比对导出图与参考模型。图 2-7 是工作台导出的整帧画面。



图 2-7 仿真工作台导出的整帧画面

2.3.2 软件各模块介绍

软件从顶层到底层分十块：启动、主循环与状态机、输入、对象池、弹幕更新、行桶构建、渲染、显示驱动、音频、遥测。下面按调用顺序说明各块。

（1）启动与初始化。流程：diag_init → gpu2d_init → 清帧缓冲 → gamepad_init → entities_init → gpu2d_asset_copy 拷走 16384 字节资产 ROM → 解码精灵、弹幕图块、字模、背景与特效帧 → gpu2d_flush_dcache。输入是片上 BRAM 中的固件镜像和 16 KiB 资产 ROM。精灵表 ART_COUNT 为 26，其余贴图、关卡背景和字模在开机时写入 DDR。

（2）主循环与状态机。main() 的 for(;;) 每圈先调用 audio_stream_service()。菜单和 CPU 与 FPGA 对比界面在主循环内各自绘制。关卡和压力关按

sim_acc 调用 game_update()，一圈 0 到 4 次，再调用 game_render_pass() 出图。screen_state 由主循环按手柄改写。game_update() 更新实体池，telemetry_write() 写出 tlm[]。

（3）输入。gamepad_read() 读取 I2C 锁存的摇杆和按键。摇杆收成中、低、高三档，五个按键分别记入 btn_a、btn_b、btn_x、btn_y 和 btn_center，没有合并成一个 buttons 字段。按键边沿在 main() 里判断。输出是三档摇杆值和五个独立按键位。

（4）对象池。alloc_bullet_range() 在给定区间内找最低空槽，找不到返回空指针。spawn_ebullet() 等生成函数遇到空指针直接返回，不计入 DROP。敌弹池地址是 MM_EBULLET_BASE，容量 MM_EBULLET_COUNT 为 30720。输出是 bullet_t 指针；压力关另维护活动弹计数。

（5）弹幕更新。压力关中整数速度的敌弹由分桶协处理器移动，bullets_update() 跳过这些记录。留在 CPU 上的普通弹按整数速度前进。带 EF_SUBPIXEL 的弹使用 4 位小数，即 Q4，不是 16.16 定点。出界或寿命到期的槽被清掉，随后做命中、得分和粒子处理。

（6）行桶构建。默认编译由 bucket_build_start() 刷新数据缓存并启动协处理器，bucket_build_finish() 等待结束并读回 DROP。每组容量 1024 条。aim_index() 与 bucket_put_run() 只在默认关闭的软件对照路径中使用。输入是弹池和玩家坐标，输出是 DDR 中的 MM_BUCKET_TABLE 与 DROP 计数。

（7）渲染。game_render_pass() 先调用 clear_frame()，再进入 game_render()。damage_restore() 按 96 行、每行 8 像素的账本把背景补回。贴图和半透明分别调用 gpu2d_copy() 与 gpu2d_alpha_blit()，文字由 draw_text() 绘制，激光、光束、天气和冲击环写入扫描层寄存器。翻页写 GPU2D_REG_FRAME_NEXT，一次更新左右扫描基址。gpu2d_swap_buffers() 仍在驱动中，主循环不调用它。

（8）显示驱动。流程：gpu2d_fill、copy、add、alpha_blit 把地址、宽高、跨距、颜色或 Alpha 写进 APB 寄存器，写 GPU2D_REG_START 后轮询状态位。输入是地址、尺寸、跨距、颜色、Alpha 与颜色键；输出由 GPU2D_REG_STATUS 的 BUSY、DONE、CFGERR、TIMEOUT 给出。

（9）音频。流程：audio_stream_init() 用 0xAB 唤醒 SPI Flash 并校验

BGM1 头；主循环调 audio_stream_service()，读 PCM_STATUS 水位，低于 2048 字节就补最多 1024 字节。输入是 Flash 中 0x00200000 起的 PCM；输出是 I2S FIFO 字节流。

（10）串口调试功能。telemetry_write() 把 tlm[] 写入 MM_TELEMETRY 的 64 个字。diag_window() 每 32 圈主循环在 UART 上打印一组键值。帧率不读这块内存：硬件 OSD 和关卡 HUD 都读 APB 上的 BCD 帧率寄存器。输入是本圈的 tlm 与 BitBLT 计数，输出是 DDR 遥测块和 UART 日志。

第三部分　完成情况及性能参数

3.1 整体介绍

系统实物由一块 Ti60F225I3 开发板、一个 I2C 手柄、一块 MAX98357A 功放模块与一只 4 Ω扬声器、一台 1366×768@75Hz 的 HDMI 显示器以及一根 USB 线组成。开发板通过 USB 供电，同时承担 JTAG 下载与 UART 通信；手柄接在 40 针扩展口的 I2C 两根线上；功放模块的 VIN 接板卡 5 V，三根 I2S 信号线接到 J7.34、J7.38、J7.40；HDMI 直接连显示器。

上电后三秒内完成 DDR3 校准与资产解码，屏幕先出现菜单。进入关卡、压力关和对比界面的操作见 1.2。整机在 5 V 侧的工作电流约 0.35A，板卡温升约 20 ℃，连续运行两小时未出现画面异常。图 3-1 与图 3-2 为系统实物照片，分别从正面和斜 45° 拍摄。

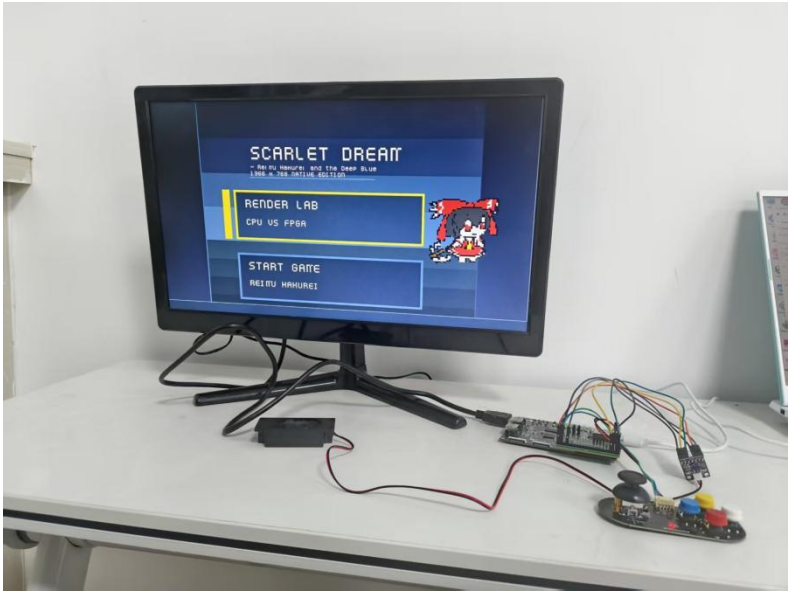


图 3-1 系统正面照片



图 3-2 系统斜 45° 照片

3.2 工程成果

工程成果分为电路与软件两部分。电路方面给出板级系统的实物照片与接线记录，软件方面给出工作台导出的运行界面。

3.2.1 电路成果

硬件部分的成果是一块完整的系统：FPGA 经板载 FT4232HL 与 PC 相连，一根线完成供电、下载和串口；40 针扩展口引出 I2S 三线与 I2C 两线，接功放与手柄；HDMI 输出 1366×768、75 Hz 的画面。板上 LED 与按键沿用原厂定义。KEY0 与 KEY2 按下后都回到主菜单，操作见 1.2。

DDR3 在 400 MHz 物理时钟下校准一次通过，cal_done 拉高后系统复位释放，没有出现需要手动复位才能启动的情况。音频链路上电后先听到背景音乐，再进入游戏可听到射击、击杀与大招三种不同音色。

图 3-3 为开发板与外围模块的接线实物照片。

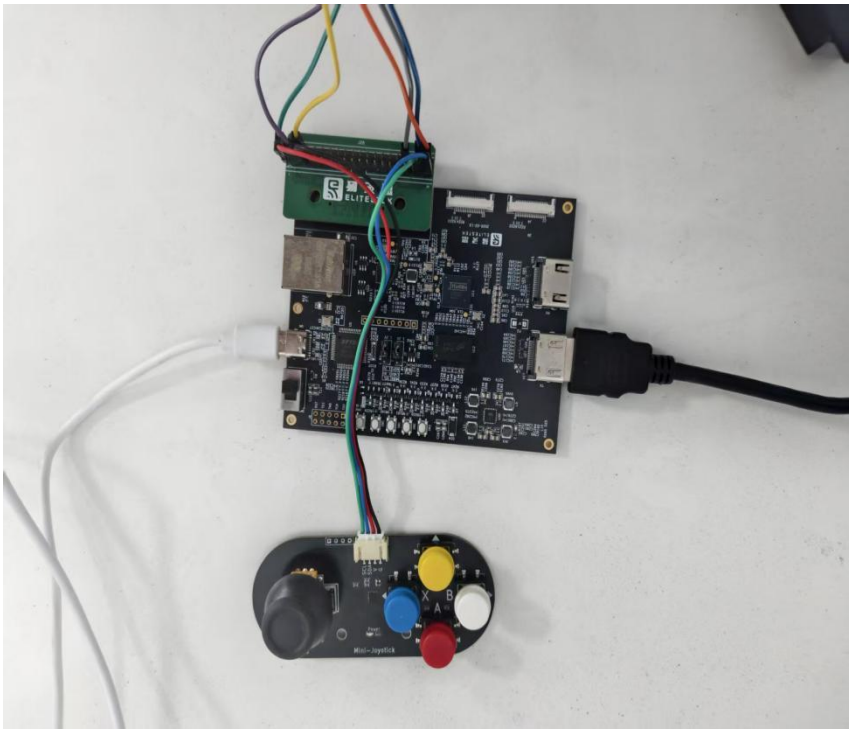


图 3-3 开发板接线图

3.2.2 软件成果

软件成果是一份 64 KiB 的固件与一套仿真工作台。固件在板上稳定跑完三关流程与压力场景，画面由硬件 BitBLT 与扫描层合成，对比界面的硬件 OSD 左右各显示一路帧率；压力关边栏显示 SPRITES 和 DROP；关卡 HUD 只画一路帧率，读的是右路 BCD。工作台可以在没有开发板的 PC 上跑同一份固件并导出逐帧图像，用于回归比对。下面三张图由工作台导出：图 3-4 是暂停与设置界面，图 3-5 是第三关 BOSS 的激光弹幕，图 3-6 是第二关 BOSS 的双环弹幕。

仿真工作台在 simulator 目录，入口是 test_runner.py。它加载 sw/build 中的 firmware.bin，由 rv32i_soc_vm.py 的 Sapphire 模型执行同一份裸机程序，不需要开发板。模型包含 64 KiB 片上 RAM、APB 寄存器、两块帧缓冲以及弹池和贴图所在的 DDR。BitBLT 的填充、拷贝、颜色键、Alpha 混合和饱和加法按 RTL 的字节结果执行。导出图片时，scanout_model.py 把天气、弹幕、激光、光束和 HUD 叠到帧缓冲上，输出 1366×768 的 BMP 与 PNG。设置 SIM_NO_SCANOUT 后只导出帧缓冲，用来区分底图错误和扫描层错误。

字节级对照与 Verilog 测试台使用同一份激励。blit_ref.py 对应矩形拷贝，

alpha_ref.py 对应 Alpha 混合和颜色键；两边都对结果做 FNV-1a 64 位摘要，摘要一致才算通过。bucket_model.py 复现敌弹的整数位移、越界回收、31×31 危险框候选和按 8 行一组的桶表，它不模拟 AXI 时序。这些脚本从 sw/driver/memory_map.h 读取地址，固件搬迁显存后，工作台不会继续检查旧地址。

test_runner.py 的 suite_catalog() 现有 28 个场景，分成渲染、菜单与操作、回归、三关技能、压力关和激光清场等组，可以串行运行，也可以每个场景一个进程。场景读取遥测块中的生命、得分、关卡和精灵数，并在需要时保存当前帧。图 3-4 至图 3-6 即由该工作台导出。build_native.py 可以把同一语义编成 C++ 扩展；删除该扩展后自动退回 Python 模型。



图 3-4 暂停与设置界面

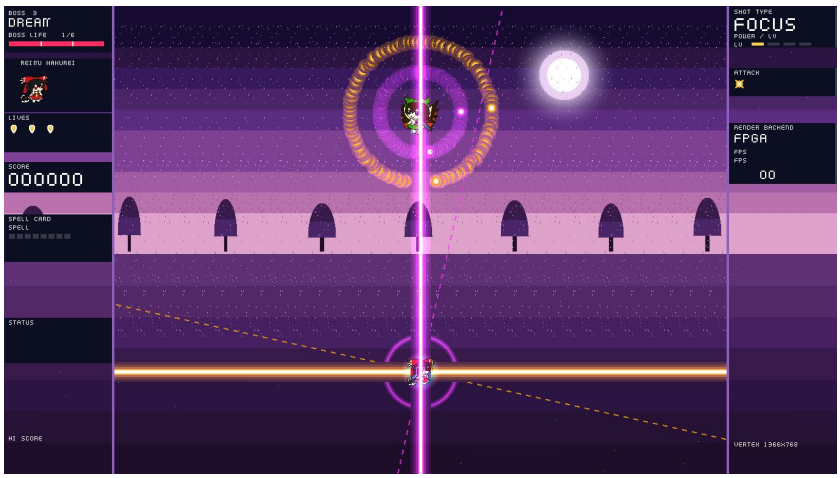


图 3-5 第三关 BOSS 的激光弹幕

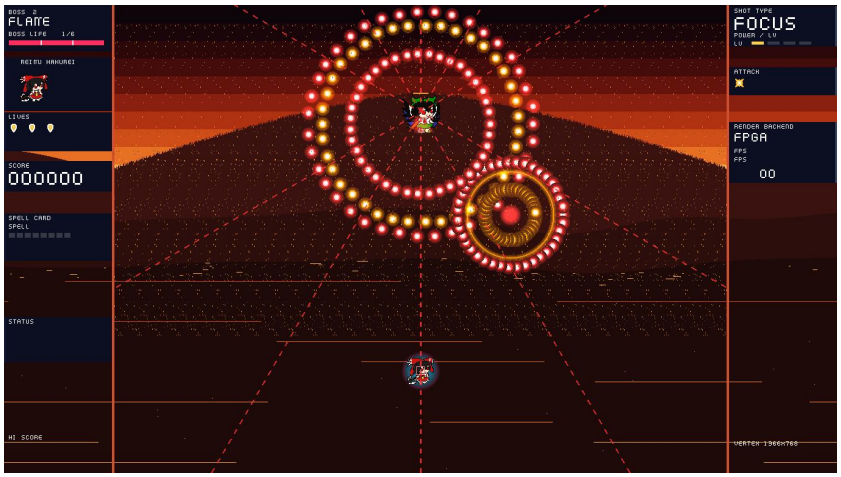


图 3-6 第二关 BOSS 的双环弹幕

3.3 特性成果

功能方面，系统实现菜单、三关流程、BOSS 多阶段弹幕、暂停与设置界面、压力场景与 BitBLT 自检，全部通过手柄操作。可量化的指标如下。

（1）帧率。同一压力场景（192 个活动精灵、满屏弹幕）在纯软件渲染下平均 25 FPS，启用硬件 BitBLT 与扫描合成后稳定 75 FPS，提升约 3 倍。

（2）弹幕规模。压力关的 LOAD 从 192 起、按 128 调整，固件上限为 30000 发。分桶协处理器完成敌弹的整数位移、越界回收和 31×31 危险框候选筛选，并按 8 行一组登记桶表。某一组写满 1024 条时记一次 DROP，该弹仍留在弹池中。

（3）显存写入。20000 发弹幕在扫描期间实时合成，写入帧缓冲的像素为 0 字节，背景修补的开销随之归零。

（4）资源占用。XLR 58982 / 60800（97.01%），内存块 256 / 256（100%），DSP 107 / 160（66.88%），触发器 27098，SRL8 1073。

（5）时序。core_clk 周期 10 ns、像素时钟 11.667 ns、DDR3 物理时钟 2.5 ns、TMDS 串行时钟 2.333 ns 全部收敛，全设计最差建立裕量 +0.364 ns。

（6）固件。当前链接镜像为 64180 字节，结束于 0xF900FAB4。0xF900FD00 至 0xF9010000 的 768 字节留给栈，镜像距栈底还有 588 字节。

图 3-7 与图 3-8 是同一帧在关闭与开启 Bloom 滤镜时的对比。光晕由 SRL8 移位寄存器链在像素流水线内完成，不占 CPU 周期，也不增加 DDR 消

耗。

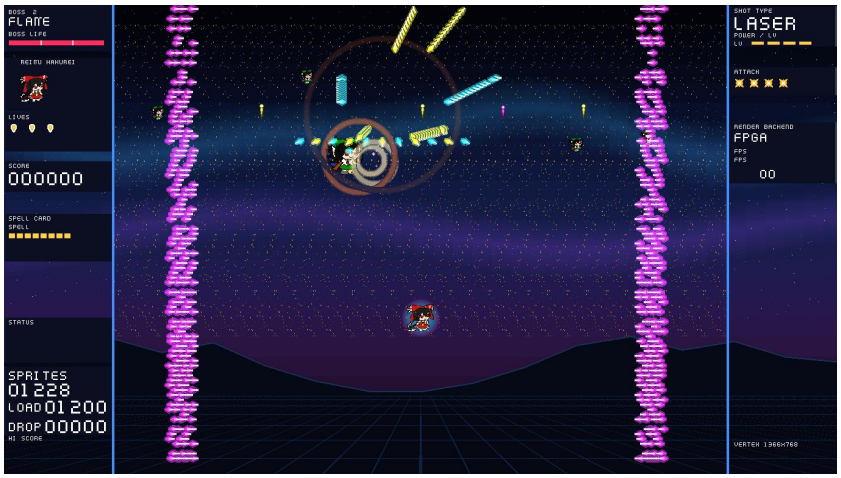


图 3-7 关闭 Bloom 滤镜的画面

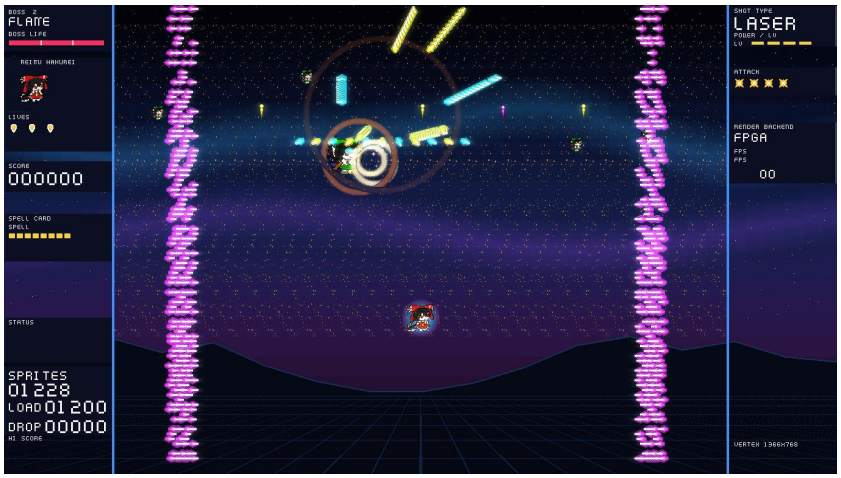


图 3-8 开启 Bloom 滤镜的画面

第四部分 总结

4.1 可扩展之处

系统还可以从三个方向继续做。一是行桶当前为 96 组、每组 1024 条、每条 4 字节。单表 384 KiB，双表 768 KiB，从 0x04800000 起到 0x048C0000。同一组超过 1024 条时，多出来的登记计入 DROP。后续可以把组内槽位改成两级索引，减轻同一扫描带挤满时的丢弃。二是把 BitBLT 的 Alpha 混合从每像素回读背景改成行缓冲合并，降低高光叠加时的读带宽。三是把资产从 DDR 一次性预载改成按需流式解压，当前行桶双表末端

为 0x048C0000，合 72.75 MiB。memory_map.h 仍为 BGM 保留从 0x05000000 起的 8 MiB，末端 0x05800000，合 88 MiB。当前 BGM 从 SPI Flash 读取，不写这块 DDR。256 MiB 里 88 MiB 之后仍是空的。在这套总线之上，增加键盘输入、更多关卡、在线固件升级都不需要改动时序约束，新的图层只要在扫描端加一级流水即可接入。

4.2 心得体会

这个题目最初看起来只是“把画图搬到硬件上”，真正做下来，难点几乎都不在画图本身。第一版把每一颗子弹都当成一张小图，用 BitBLT 逐颗贴进帧缓冲，逻辑很直白，压力场景却只有 25 FPS。剖析之后才看清原因：子弹本身不大，代价在于每贴一颗就要读背景、写前景，一帧下来显存读写量随弹数线性上涨，背景修补还要再读一遍。后来把弹幕改成“不写显存”：固件只往 DDR 里写一张 8 行一组的行桶表，扫描线走到哪一行，硬件现场把这一行该显示的子弹合成上去。同一场景的帧率从 25 跳到 75，而画面上的弹幕一颗没少。

第二个坑是画面撕裂。板子刚跑起来时偶尔出现横向裂开、整行错位，一开始怀疑 DDR 时序，后来用探针逐段排查，定位到四处独立的原因：写守卫把超时倒计时并进了准入条件，BitBLT 大面积写入时放行了本该压住的写事务；齿轮箱在 FIFO 读空时计数器仍在跑，数据回填后整行像素偏移 1 到 3 个像素；场消隐脉冲只有一拍，跨到 100 MHz 域时偶发丢采样，导致行地址没归零；读基地址在模块内部另存了一份影子寄存器，翻页时两边时序不一致。四处都改掉之后撕裂才彻底消失。这件事让我明白，跨时钟域的问题不能靠“多试几次”解决，要把每个域里的状态摆在同一张纸上比对。

第三个坑是地址。内存映射最早散落在各处，改一个功能就挪一块空间，结果三个不同用途的缓冲悄悄叠在一起：角色倾斜的两个姿势、冲击帧和大招的激光柱，读到的都是视差背景的像素。后来把所有地址收进一个头文件，写了一个脚本从源码重新推导每一段的范围，发现重叠就让构建失败，这类问题才不再复发。

还有一个体会是“量出来再改”。性能优化时我一度凭直觉去改渲染循环，收效很小；改成在虚拟机里采指令、在板子上读周期计数器之后，才发现一帧里有一半时间耗在构桶和子弹更新上。照着数据去改，帧率才慢慢爬上来。20000 发

的那一轮也是同样的过程：先量到写回是一个事务一颗弹，改成突发写回之后耗时从 9.86 ms 降到 6.825 ms。

最后是空间。固件只有 64 KiB，每次加功能都要先看还剩多少字节。当前链接镜像结束于 0xF900FAB4，共 64180 字节，距栈底 0xF900FD00 还有 588 字节。这种限制逼着人去想“这段代码真的需要吗”，也让我对嵌入式系统的分寸感有了具体的认识。整个项目做下来，最大的收获不是某个模块写得多巧妙，而是学会了把现象变成数字、把数字变成原因、再把原因变成一行改动的顺序。

第五部分　参考文献

[1] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 20191213[S]. 2019.

[2] Efinix Inc. Sapphire SoC User Guide[Z]. 2025.

[3] emakefun. Mini-Joystick 模块使用说明[EB/OL]. https://emakefun.github.io/emakefun-docsify/.

[4] 易灵思. Titanium 系列 FPGA 数据手册[Z]. 2025.

[5] 上海爱丽丝幻乐团. 东方 Project 二次创作指南[EB/OL].

[6] 李洪革. FPGA 嵌入式系统设计与应用[M]. 北京: 机械工业出版社, 2020.